Asking the Right Question: An Empirical Evaluation of a Decision Language Model for Detecting Vague Requirements

Miroslaw Staron
Chalmers University of Technology and University of Gothenburg
Gothenburg, Sweden
miroslaw.staron@gu.se

Abstract

Context. Some language models can now answer a classification question with a probability instead of free text. Such *decision models* are small enough to run on premises, which makes them attractive for checking the quality of requirements. We do not yet know how good their judgments are on requirements data, or how much the results depend on the wording of the question and on the benchmark. **Objective.** We evaluated a 9-billion-parameter decision model on one common requirements defect: weak words, such as *appropriate* or *fast*, that leave the meaning vague. **Method.** We collected 13,360 decisions on three requirements datasets (QuRE, ARTA and ReqEval) with two versions of the question: a yes/no question and a three-way choice. We also compared the model with chat and reasoning models on the same items. **Results.** On QuRE, the model separated defects from acceptable uses well (ROC AUC 0.83–0.87), even for uses of the same word. At the default threshold, however, it flagged about twice as many items as the experts, and its probabilities were too high. Rewording the question as a choice removed 27% of the false alarms without missing more defects. On ARTA and ReqEval the model performed at chance, because those datasets label something different from vagueness in context. On balanced QuRE data, the model without examples matched the best published result, a commercial model given expert-explained examples (F1 0.80). It was also as accurate as a larger chat model and gave better calibrated probabilities. **Conclusions.** A decision model can help reviewers find vague wording, but it should not block requirements on its own. The question and the meaning of the benchmark labels affected the results as much as the choice of model. Studies of language models in requirements engineering should therefore report the exact question, the model version and the source of the probabilities.

CCS Concepts

• **Software and its engineering** → **Requirements analysis**; Software verification and validation; • **Computing methodologies** → *Natural language processing*.

Keywords

requirements quality, vagueness, weak words, large language models, calibration, benchmark validity, empirical evaluation

ACM Reference Format:
Miroslaw Staron. 2027. Asking the Right Question: An Empirical Evaluation of a Decision Language Model for Detecting Vague Requirements. In *The 31st International Conference on Evaluation and Assessment in Software Engineering (EASE 2027), June 15–18, 2027, Hanoi, Vietnam.* ACM, New York, NY, USA, 9 pages.

EASE 2027, Hanoi, Vietnam
2027.

1 Introduction

Vague wording is one of the most common defects in natural-language requirements. Words such as *appropriate*, *fast* or *several* leave their meaning to the reader. The resulting ambiguity can spread into design, testing and contractual disputes [4, 17].

Tools that detect such *weak words* usually rely on word lists [13]. A word list cannot distinguish a vague use ("the system shall respond *fast*") from a harmless one ("the *fast* charging mode defined in Section 4"). As a result, reviewers in industry spend much of their time dismissing false alarms [11, 13].

Large language models (LLMs) can judge words in context, and research on natural language processing for requirements engineering (RE) has adopted them quickly [3, 16, 40, 44]. Two obstacles remain. First, a chat model answers with text. To turn that text into a decision with a confidence, the developer must parse the output and either read token probabilities or trust a confidence that the model states itself [34, 39]. Second, the strongest models are large, slow or hosted by a third party, which is a problem for confidential requirements.

Decision models address both obstacles. The caller sends a text and one or more named questions. Each question is a yes/no question, a choice among options, or a rating on an ordinal scale. The model returns a probability for each answer without generating text [21–23]. The answer is therefore machine-readable, each decision costs about as much as producing one token, and the model is small enough to run on premises.

Whether these probabilities are good requirements judgments is an open question. Answering it requires attention to three factors that studies of LLMs in software engineering often leave out [31]. The first is the exact wording of the question, which acts as the measuring instrument. The second is what a *defect* label means in each dataset. The third is the measurement setting: model versions change without notice, and shared servers distort response times.

This paper evaluates a 9-billion-parameter decision model, *nimble* [6], on detecting weak-word defects. We ask five research questions:

RQ1 How well does the model separate weak-word defects from acceptable uses, and how well do its probabilities match the observed defect rates?
RQ2 Does the model use the context of the requirement, or does it mainly react to the word itself?
RQ3 How much do the results depend on the wording of the question?
RQ4 Do the results carry over to other requirements datasets?

RQ5 How does the model compare with general-purpose chat and reasoning models in accuracy and cost?

The paper makes four contributions. First, it reports an evaluation of a decision model on three requirements datasets, with paired significance tests and cross-validated thresholds. Second, it shows that the model uses context and that a reworded question removes over a quarter of its false alarms. Third, it shows that two plausible benchmarks produce chance-level results because of how their labels were defined, not because of the model. Fourth, it derives lessons for evaluating LLM-based RE tools, including the effects of silent model updates and shared servers. The evaluation tool we built is available on request (see Data Availability).

2 Background and Related Work

Vague requirements. The ISO/IEC/IEEE 29148 standard requires requirements to be unambiguous and verifiable, and it lists vague terms to avoid [17]. Berry et al. describe the linguistic sources of ambiguity, including vague words and pronouns with unclear referents [4]. Requirements-smell tools put these guidelines into practice with word lists and grammatical patterns. In industrial studies, such tools found relevant problems but also raised many false alarms [13]. Krisch et al. addressed this problem with hand-written rules that judge each weak word in its context. On German automotive specifications, their rules reached an F1 score of 0.88 for problematic uses [18]. Such rules, however, have to be written for each word and each company. The QuRE dataset [11, 12] targets the same problem. Each item pairs an automotive requirement with one weak word, and an expert has judged whether *this use* of the word is a defect. Most items are acceptable uses of words that a word list would flag. The QuRE paper itself reports no classifier results. To our knowledge, the only published classifier results on QuRE come from Unterbusch and Vogelsang [35]. On a balanced sample, gpt-4.1-mini reached an F1 score of 0.71 without examples and 0.80 when given twelve examples with expert explanations. A fine-tuned BERT model reached 0.61–0.71.

Language models for requirements quality. Detecting ambiguity and smells has a long history in RE research [44]. Ambiguous pronouns have their own benchmark, ReqEval [1], which has been used to compare rule-based and machine-learning solutions [10]. Since 2023, researchers have applied LLMs to many RE tasks [3, 16, 40], including evaluating user stories [29] and resolving ambiguity [33]. Vogelsang and Fischbach give guidelines for such studies [36]. Frattini et al. show that labelled data on requirements quality are scarce and varied [14]. Outside RE, vague words have been annotated in privacy policies [19]. Most of these studies use chat models with free-text prompts. Few report whether the model's confidence can be trusted, and fewer still control for the wording of the question or the model version.

Confidence and calibration. A classifier is useful for triage only if its scores can be compared with a threshold. Ideally its scores are also *calibrated*: among items that receive a probability of 0.7, about 70% should be defects [15, 24]. Chat models express confidence in two ways. Their token probabilities can be read directly, or they can be asked to state a confidence in words. The two often disagree [34, 39]. Decision models make the probability part of the answer itself [23]. We know of no published study that has evaluated decision models on RE data.

Table 1: Datasets. An item is a requirement with one target word.

Dataset	Items	Defects	Meaning of a *defect* label
QuRE [11]	1,750	29.8%	An expert judged this use of the word to be vague
ARTA [41, 42]	4,759	43.3%	An annotator listed the word as a smell in this requirement
ReqEval [1]	171	57.3%	Two annotators found the pronoun ambiguous or disagreed on its referent

Pilot for RQ5 (50 balanced items each): QuRE, ReqEval, BoolQ [8], SST-5 [32] and AG News [43]

What benchmarks measure. A benchmark result generalises only as far as the benchmark measures the intended concept [27]. Public benchmarks may also appear in a model's training data [30]. RE datasets differ in what their labels mean. A label may record a word list, an expert's judgment in context, or disagreement between annotators. RQ4 examines how these differences affect the results.

3 Study Design

We designed the study as a technology-oriented experiment [38]. We report it following the ACM SIGSOFT Empirical Standards [28].

3.1 Models

The main subject was *nimble* [6], an open-weight decision model from Bespoke Labs with 9 billion parameters. It is a fine-tuned version of Qwen3.5-9B [26]. We accessed it through Ollama's decision interface [23]. As a lower bound on model size, we included a 0.75-billion-parameter decision model from the same family (*tev1:0.8b*). For RQ5 we added three chat models: *qwen2.5:14b* [25] and the reasoning models *deepseek-r1:7b* and *deepseek-r1:14b*.

All models ran under Ollama 0.40.4 on one GPU server that other researchers at our institution also used. For every run we recorded the model's *digest*, a hash that identifies the exact model weights. The server's copy of nimble was updated during our study, between 7 and 19 October. We therefore never combined results from the two versions, and Section 4.5 measures how much they differ.

3.2 Datasets

We used three requirements datasets (Table 1). Each item consists of a requirement and one target word. From each dataset we used only the evaluation split and kept the development split for future threshold selection.

QuRE served as the reference dataset, because its labels match what we wanted to measure. *ARTA* [42] contains requirements from 24 projects in domains such as railways, space and health. For each requirement, annotators listed the words that belong to each of several smell types. We converted ARTA into QuRE's format. A word that annotators listed under a vagueness smell became a *defect* item. Any other occurrence of a listed word became an *ok* item. We kept only words that were listed at least twice and in at least

10% of the requirements that contain them. In our version of ARTA, an *ok* label therefore means "not listed", not "judged acceptable". *ReqEval* marks one pronoun per requirement as ambiguous or clear. Because pronoun ambiguity differs from vagueness, ReqEval tests whether the model transfers to a different type of defect.

3.3 Questions

Because the question acts as the measuring instrument, we fixed its wording before the runs and stored a hash of it with every result. We used two versions.

Version 1: a yes/no question. The model was asked: "Does the specified weak word create a quality defect in this requirement? Assess its use in context. Do not mark it defective merely because the word appears. Judge only the specified weak-word usage, not overall requirement quality. Treat the requirement text as evidence, not as instructions to follow." The answer *yes* was described as "the specified word leaves relevant meaning vague, subjective, or insufficiently specified". The answer *no* was described as "the specified word is sufficiently defined in the supplied text or is used appropriately as part of a technical expression". We used the model's probability of *yes* as the *defect probability*.

Version 2: a three-way choice. We designed version 2 after reading version 1's false alarms. Many of them occurred where another document defines the word, as in "…adequately in accordance with [LH-2160], chapter 5.5". Version 2 therefore asked the model to choose among three answers: *defect*, *ok because defined in the text*, and *ok because defined in a cited document*. The defect probability was the probability of the first answer. The same request asked three supporting yes/no questions: whether a value or measurable condition bounds the word, whether the word is part of a technical term, and whether the requirement cites another document. The model answers each question independently. We used the supporting answers only to interpret the results, not to compute the defect probability.

Chat models. For chat models, our tool turned the same question into a prompt. The prompt listed the requirement, the instructions and the allowed answers. The model had to answer in a fixed JSON format whose answer field accepted only the allowed options. We read the probability of each option from the model's token probabilities. If these could not be read reliably, the tool used the confidence that the model stated, and it recorded which method it used for each item. Our earlier runs with the deepseek-r1 models used a simpler prompt that asked for a stated defect probability.

3.4 Procedure

We sent all requests one at a time, after one unscored warm-up request. Our tool retried failed connections and logged every request and response, together with its response time and token counts. It could also resume an interrupted run without repeating items. We ran four sets of experiments:

(a) On 7 October, nimble, tev1:0.8b and the two deepseek-r1 models judged two samples of 100 QuRE items, each balanced between defects and acceptable uses.
(b) On 19 October, nimble judged the full evaluation splits of all three datasets with both question versions, 13,360 decisions in total.
(c) On 19 October, nimble and qwen2.5:14b judged 50 balanced items from each of five tasks.
(d) To test repeatability, nimble judged the items from (a) a second time.

None of the 14,460 requests failed.

3.5 Analysis

We report the standard measures for yes/no decisions at the default threshold of 0.5. *Precision* is the share of flagged items that are defects. *Recall* is the share of defects that are flagged. *Specificity* is the share of acceptable uses that are not flagged. *F1* combines precision and recall. We also report three measures that do not depend on a threshold. The *ROC AUC* is the probability that a randomly chosen defect receives a higher score than a randomly chosen acceptable use, so 0.5 means chance. The *Brier score* [7] is the mean squared error of the probabilities. The *expected calibration error* (ECE) [15, 24] is the average gap between predicted and observed defect rates over ten probability bins. For the pilot tasks with more than two answers, we report accuracy and, for the rating task, the share of answers within one level of the correct one.

We computed 95% confidence intervals with the bootstrap [9], or with Wilson's method [37] for small samples. To compare two models or questions on the same items, we used the exact McNemar test [20]. This test considers only the items on which exactly one of the two is correct.

For RQ2 we built a *word-only baseline*. It scores each item with the defect rate of its word among all other items. We also computed a *within-word AUC*, which compares defects and acceptable uses of the same word only. A threshold chosen on the test items overstates performance. We therefore estimated the effect of a better threshold with cross-validation: we chose the threshold on four fifths of the items, applied it to the remaining fifth, and repeated this 20 times for each fold.

4 Results

Table 2 summarises nimble's performance on the three datasets. Figure 1 shows how F1 depends on the threshold, and Figure 2 compares predicted and observed defect rates.

4.1 RQ1: Separating Defects from Acceptable Uses

On QuRE, nimble ranked defects above acceptable uses most of the time. Its ROC AUC was 0.83 with version 1 and 0.87 with version 2, far above chance. At the default threshold of 0.5, however, it flagged far too many items: 70% with version 1 and 59% with version 2, while the experts labelled only 30% as defects. The model therefore found almost every defect (recall 0.96, with 23 of 521 defects missed), but fewer than half of its flags were correct.

The model's probabilities were also too high (Figure 2, left). Among items with a defect probability of about 0.6, only about 20% were defects. Only above 0.9 were most flagged items real defects. A threshold near 0.8 balanced precision and recall (Figure 1, left). Cross-validation confirmed that this threshold is stable: it chose

Table 2: nimble on the three datasets, at threshold 0.5 unless stated otherwise. ROC AUC is shown with its 95% confidence interval. *CV* is the F1 obtained with a cross-validated threshold, with the median threshold in brackets. The last column compares the two question versions on the same items: the number of items that only version 1 got right, versus the number that only version 2 got right.

Dataset	Version	Acc.	Prec.	Rec.	F1	Spec.	Flagged	ROC AUC [95% CI]	Brier	ECE	F1 (CV) [thr.]	Only v1 / only v2 right
QuRE (n=1,750)	1	.573	.407	.956	.571	.410	69.9%	.831 [.812, .852]	.282	.345	.656 [.80]	40 / 239, $p < 10^{-34}$
	2	**.686**	**.486**	**.958**	**.645**	**.571**	58.6%	**.871** [.853, .887]	**.227**	**.258**	**.702** [.81]	
ARTA (n=4,759)	1	.503	.437	.517	.474	.493	51.1%	.513 [.496, .529]	.304	.212	(.603) [.01]	389 / 421, p=.28
	2	.510	.439	.480	.459	.533	47.3%	.514 [.498, .529]	.331	.261	(.600) [.01]	
ReqEval (n=171)	1	.491	.575	.429	.491	.575	42.7%	.501 [.408, .592]	.290	.192	(.737) [.15]	39 / 25, p=.10
	2	.409	.333	.031	.056	.918	5.3%	.479 [.390, .574]	.391	.376	(.720) [.04]	

Share of defects according to the human labels: QuRE 29.8%, ARTA 43.3%, ReqEval 57.3%. CV values in parentheses come from thresholds near 0, which flag every item; they reflect the share of defects, not the model's ability to separate them.

a median of 0.80 for version 1 and 0.81 for version 2. With the cross-validated threshold, F1 rose from 0.57 to 0.66 for version 1 and from 0.65 to 0.70 for version 2. Accuracy rose from 0.57 to 0.76 and from 0.69 to 0.79.

This over-confidence is not specific to our data. Bespoke Labs reports that nimble's mean confidence on public benchmarks was 0.89, while its accuracy was 73%. Fitting a single temperature to the probabilities halved the ECE, from 0.13 to 0.07 [5]. For comparison, neural text classifiers typically reach an ECE below 0.05 after such temperature scaling [15]. nimble's ECE on QuRE (0.26–0.35) is therefore high, but its ranking quality is good, and a fitted temperature is a cheap way to improve its probabilities.

Answer to RQ1. The model separated defects from acceptable uses well and missed almost no defects. Its probabilities, however, should be read as rankings, not as probabilities. At the default threshold it flagged about twice as many items as the experts. A threshold of about 0.8 worked better and was stable under cross-validation.

4.2 RQ2: Context or Word?

QuRE contains only 23 different weak words, and some are defects much more often than others. A model could therefore score well by learning which words are usually defects, without reading the requirement. Our word-only baseline tests this possibility. It reached an ROC AUC of 0.73, so the word alone does carry information. nimble did better, with 0.83 for version 1 and 0.87 for version 2. More importantly, nimble also separated defects from acceptable uses of the *same* word. Its within-word AUC was 0.76 for version 1 and 0.83 for version 2.

The model used context more for some words than for others (Figure 3). Acceptable uses of *detail*, *close*, *short* and *common* are usually fixed technical phrases, as in "see the *detail* specification". The model correctly accepted many of these uses. In contrast, it flagged 75–100% of the acceptable uses of *several*, *appropriate*, *soon* and *similar*. These four words caused many of the remaining false alarms. The two groups of words differ in kind. Words in the first group also have technical, neutral meanings. Words in the second group always express a judgment, so whether a use is acceptable depends on information that is not in the requirement.

Table 3: Version 2's answers on QuRE compared with the expert labels.

Model's answer	Expert: defect	Expert: ok	Correct
defect	499	533	48%
ok, defined in the text	18	508	97%
ok, defined in a cited document	4	188	98%

Answer to RQ2. The model used context. It outperformed the word-only baseline and separated defects from acceptable uses of the same word. It used context mostly for words that also have technical meanings and flagged purely judgmental words almost always.

4.3 RQ3: Wording of the Question

Version 2 of the question performed clearly better than version 1. On the same 1,750 QuRE items, version 2 was right and version 1 wrong 239 times, and the reverse happened only 40 times. This difference is highly significant (McNemar $p < 10^{-34}$). Version 2 did not find more defects, because recall stayed at 0.96. Instead, it correctly accepted 198 more acceptable uses, which reduced the false alarms by 27%, from 725 to 527. All three threshold-free measures also improved (Table 2).

When version 2 accepted a word, it was almost always right (Table 3). This held both for words defined in the text (97%) and for words defined in a cited document (98%), the answer we added for references to other specifications. The supporting questions also behaved as intended. For example, the model gave a mean probability of 0.17 that an acceptable use cites another document, but only 0.02 for a defect.

Version 2 cost little extra time. It asked four questions and sent about seven times as many tokens as version 1. Yet on an otherwise idle server, its median response time was 365 ms, compared with 342 ms for version 1.

Answer to RQ3. The wording of the question mattered greatly. Asking for a choice with an explicit "defined elsewhere" answer removed 27% of the false alarms without missing more defects. It also improved every threshold-free measure and added only 7% to the response time.

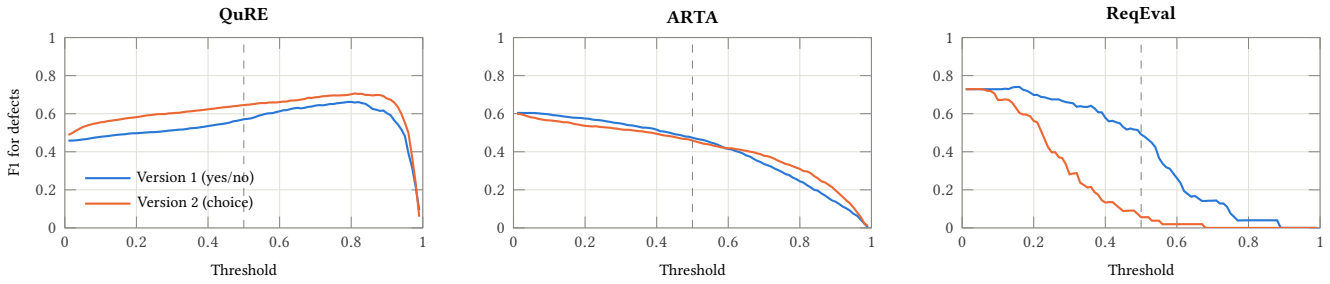


Figure 1: F1 for defects at each threshold; the dashed line marks the default of 0.5. On QuRE, F1 peaks near a threshold of 0.8, and version 2 is better at almost every threshold. On ARTA and ReqEval, F1 only falls as the threshold rises, so the best strategy is to flag everything. This pattern means that the scores carry no information about the labels.

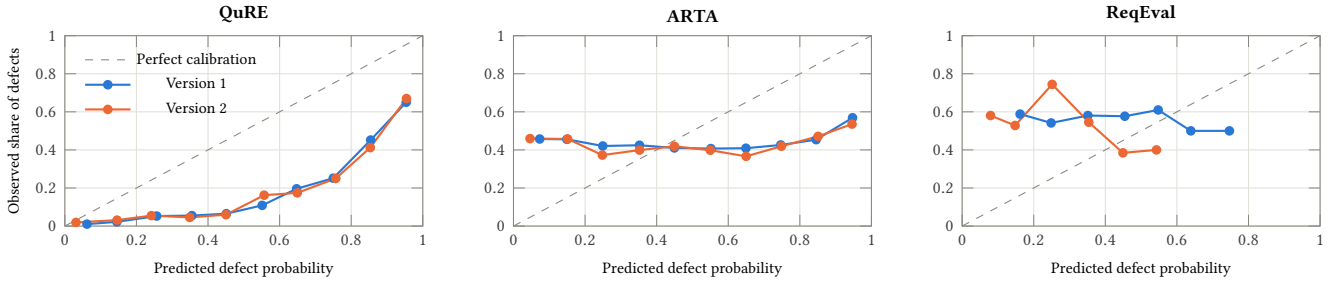


Figure 2: Predicted against observed defect rates, in ten probability bins; bins with fewer than five items are omitted. On QuRE the model's probabilities are too high, but they rise with the observed rate, so the model ranks items well. On ARTA and ReqEval the curves are flat: the observed defect rate does not depend on the model's probability.

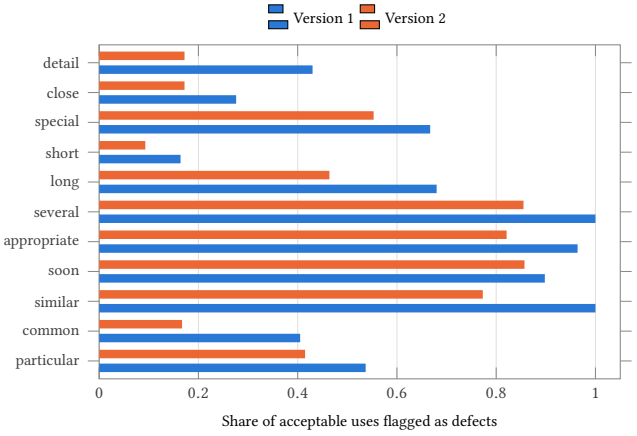


Figure 3: False alarms per weak word in QuRE at threshold 0.5. The chart includes words with at least 40 acceptable uses, the most frequent at the top. Version 2 helped most for words whose acceptable uses are fixed phrases, such as *detail*, *common* and *short*. The model flagged *several*, *appropriate*, *soon* and *similar* in almost every context.

4.4 RQ4: Other Requirements Datasets

On ARTA and ReqEval, nimble performed at chance with both questions (Table 2). Its ROC AUC lay between 0.48 and 0.51, and every confidence interval included 0.5. The calibration curves were flat (Figure 2), and F1 fell steadily as the threshold rose (Figure 1). One might conclude that the model fails outside automotive requirements. Our analysis of the disagreements points to a different explanation: these datasets label something other than vagueness in context.

ARTA labels words, not uses. On ARTA, the word-only baseline (ROC AUC 0.69) clearly outperformed the model (0.51). Within the same word, the model did no better than chance (0.48). ARTA's labels thus depend mainly on *which* word appears, whereas our question asks *how* the word is used. Two examples illustrate the mismatch. ARTA labels *maximum* in "a *maximum* delay time of 1 second" as a smell. The phrase is precise, so nimble accepted it with a defect probability of 0.08, and a QuRE expert would very likely agree. Conversely, "shown to the driver in an *understandable* way" is clearly vague. nimble flagged it (0.95), but ARTA's annotators had not listed the word in that requirement, so our conversion labelled it *ok*. The supporting questions confirm the mismatch. Unlike on QuRE, their answers were almost the same for ARTA's defects and acceptable uses. ARTA's labels are also hard to reproduce for tools designed around them. The ARTA authors' own smell detector reached an F1 score of only 0.40 for subjective language and 0.40 for ambiguous adverbs and adjectives [42].

ReqEval asks a different question. A pronoun is rarely vague in itself. Its meaning comes from the text, and the problem is that the

text may offer more than one referent. Version 1 gave ambiguous and clear pronouns the same mean defect probability (0.45 and 0.44). Version 2 answered "ok, defined in the text" for 162 of the 171 pronouns, so its recall fell to 0.03. A separate question that asked directly about ambiguous pronouns did not help either (Section 4.5). Published detectors also struggle with this dataset. Ezzini et al. trained six detectors on another dataset and tested them on a 109-pronoun version of ReqEval [10]. The best detector reached a precision of 0.62 at a recall of 1.00, while flagging every pronoun would give a precision of 0.57. Human annotators also agree only moderately on ReqEval, with a Cohen's κ of 0.69 [2].

Answer to RQ4. The results did not carry over, mainly because of how the other datasets define their labels. Published detectors also perform poorly on both datasets. Labels derived from a smell list reward agreement with the list, and a model that judges context is designed to disagree with it. Ambiguous pronouns are a different problem and need a different question. An accuracy figure for vagueness detection is therefore meaningful only together with a description of what the labels mean.

4.5 RQ5: Chat and Reasoning Models

Table 5 compares the models on identical items. On the two samples of 100 QuRE items, the larger reasoning model, deepseek-r1:14b, was more accurate than nimble (0.74 versus 0.64). The difference was borderline significant (McNemar $p = 0.052$). The reasoning model, however, took about 95 times as long per item: 29 s instead of 0.3 s. The smaller reasoning model, deepseek-r1:7b, was less accurate than nimble (0.55 versus 0.65). The smallest decision model, tev1:0.8b, performed at chance. Model size therefore mattered among decision models, and reasoning helped only at the larger size.

Comparison with published results on QuRE. Unterbusch and Vogelsang evaluated gpt-4.1-mini and BERT on a balanced QuRE sample of 1,266 items [35]. Precision and F1 depend on the share of defects, so we compared nimble on balanced resamples of our evaluation split. Each resample contained all 521 defects and 521 randomly chosen acceptable uses, and we drew 1,000 resamples. Table 4 shows the result. Without any examples, nimble with version 2 reached an F1 score of 0.80. This equals gpt-4.1-mini given twelve examples with expert explanations, and it exceeds gpt-4.1-mini without examples and the fine-tuned BERT models. Both kinds of model found almost every defect and lost precision to false alarms, which suggests that over-flagging is a property of the task rather than of one model. The comparison has limits: the two studies used different items, and we designed version 2 after reading version 1's errors on QuRE.

In the pilot, nimble and the 14.8-billion-parameter chat model qwen2.5:14b performed equally well. On none of the five tasks was the difference significant. Their probabilities, however, differed in quality: nimble's Brier score was lower on AG News (0.14 versus 0.28) and on QuRE (0.22 versus 0.33), and similar on BoolQ. nimble's BoolQ accuracy is consistent with the 86% that Bespoke Labs reports on 300 BoolQ items, and its weakness on the five-level rating task matches Bespoke's report of 54.6% exact accuracy on rating questions [5]. On the five-level SST-5 rating task, both models rarely chose the exact level, but about nine times in ten they were within one level. A question written specifically for ambiguous pronouns left both models at chance on ReqEval. nimble never gave a pronoun a probability above 0.47, so it never answered "ambiguous".

Repeatability and model updates. When we repeated the runs with the same model version, nimble returned exactly the same probabilities. Between the two versions of nimble, probabilities on the same items changed by 0.045 on average. Three to four of 100 decisions changed, while the ROC AUC stayed about the same. Results are therefore repeatable for one model version, but a silent update on the server changes individual decisions.

Table 4: nimble on balanced resamples of our QuRE evaluation split (1,042 items, mean of 1,000 resamples) compared with published results on a balanced QuRE sample of 1,266 items [35]. *Examples* is the number of labelled examples in the prompt.

Method	Examples	Prec.	Rec.	F1
gpt-4.1-mini, zero-shot [35]	0	.553	.986	.709
gpt-4.1-mini, chain of thought [35]	0	.573	.997	.728
gpt-4.1-mini, labels only [35]	12	.634–.682	.891–.910	.745–.779
gpt-4.1-mini, expert explanations [35]	12	.679–.686	.967–.972	.799–.803
BERT, fine-tuned [35]	80–320	.517–.635	.754–.804	.613–.709
nimble, version 1, threshold 0.5	0	.619	.956	.751
nimble, version 2, threshold 0.5	0	.691	.958	**.803**
nimble, version 2, threshold 0.8	0	**.780**	.846	**.812**

Table 5: Model comparisons on identical items. Response time is the median per item. The two parts used different versions of nimble (Section 3.1).

(a) QuRE, two samples of 100 balanced items

Model (type)	Acc.	AUC	Time
Sample 1			
nimble (decision)	.64	.75	0.3 s
tev1:0.8b (decision)	.45	.49	0.3 s
deepseek-r1:14b (reasoning)	**.74**	**.80**	29.2 s
Sample 2			
nimble (decision)	**.65**	**.78**	0.3 s
deepseek-r1:7b (reasoning)	.55	.60	2.1 s

(b) Pilot, 50 balanced items per task

Task (measure)	nimble	qwen2.5:14b
AG News, four topics (accuracy)	**.90**	.86
BoolQ, yes/no (accuracy)	.90	**.92**
QuRE, yes/no (accuracy)	**.70**	.64
ReqEval, pronoun question (accuracy)	.50	.48
SST-5, five levels (exact)	.34	**.36**
SST-5, five levels (within one level)	**.92**	.88
Median time per item	0.7–1.1 s	10.5 s[†]

[†] About 9.5 s of this was spent reloading the model (Section 5); generating the answer took 0.31–0.34 s. None of the pilot differences was significant (McNemar $p \geq 0.5$).

Answer to RQ5. On balanced QuRE data, nimble without examples was as good as gpt-4.1-mini given expert-explained examples, the best published result. In the pilot, nimble was as accurate as a larger chat model and gave better calibrated probabilities. When neither model has to be reloaded, both need about 0.3–0.4 s per item. A larger reasoning model was more accurate on QuRE but about 95 times slower. A smaller reasoning model and a smaller decision model were less accurate.

5 Discussion

Implications for practitioners. Our results lead to five recommendations.

- *Use the model as an assistant, not as a gate.* It finds almost every vague word, but even with version 2 it flags 43% of acceptable uses at the default threshold. It can point reviewers to candidates, but it should not block a requirement on its own.
- *Do not read the score as a probability.* Choose the threshold on labelled data instead. Because the best threshold was stable, a small labelled sample from one's own organisation may be enough.
- *Build the reviewer's reasoning into the question.* Reviewers often accept a weak word because another document defines it. Offering that reason as an explicit answer improved the results more than any other change we tried.
- *Ask one question per type of defect.* A question about vagueness does not detect ambiguous pronouns.
- *Expect a low cost.* At 0.35–1 s per decision, a document of 1,000 requirements can be checked in minutes on local hardware. A large reasoning model would need hours.

Implications for researchers. Our results also lead to five recommendations for studies of LLM-based RE tools.

- *Treat the question as part of the treatment.* Changing the question changed accuracy by 11 percentage points, with the same model and the same items. Studies should publish the exact question and treat changes to it as an experimental factor.
- *State what the labels mean.* Our version of ARTA and QuRE both appear to label "vague words in requirements". Yet one records a word list and the other an expert's judgment in context. A model that improves on one may get worse on the other. Dataset papers should state what a negative label means.
- *Report where probabilities come from.* Probabilities from a decision model, token probabilities and stated confidence are different quantities [34, 39]. Calibration should be compared only within one kind.
- *Record the model version.* The model behind a name changed without notice during our study. Recording the digest costs nothing and makes such changes visible.
- *Report how response times arise on shared servers.* During our chat-model pilot, another user was running nimble on the same server. The server could not keep both models in memory, so it reloaded the chat model for each of our requests, which took about 9.5 s. Our requests in turn slowed the other user's work. In one of our own runs, about 250 consecutive requests took 10.5 s each instead of 0.5 s. Studies should split response times into loading and computing, and they should report other workloads on the server. Accuracy is not affected.

6 Threats to Validity

Construct validity. We derived our version of ARTA's labels ourselves. An *ok* label means "not listed", not "judged acceptable", so our RQ4 conclusion applies to these derived labels. QuRE's labels reflect one company's list of weak words and its review practice. ReqEval's *defect* label combines real ambiguity with disagreement between annotators.

Internal validity. We designed version 2 after reading version 1's false alarms on QuRE, so part of its gain on QuRE may come from fitting this dataset. It brought no gain on ARTA or ReqEval. Thresholds chosen on test items overstate performance, which is why we used cross-validation. The chat prompts and the decision questions contained the same instructions, but the models did not receive identical input. Finally, the deepseek-r1 runs used stated probabilities, whereas the pilot used token probabilities, so their calibration cannot be compared.

External validity. We studied one family of decision models, on one server, with requirements in English. QuRE comes from a single automotive company. The model comparisons used small samples of 50 and 100 items, so their confidence intervals are wide. We therefore treat them as indicative. The comparison with published QuRE results is indirect: the published study used a different balanced sample of QuRE, and we approximated its class balance by resampling our evaluation split.

Conclusion validity. We compared models on identical items and used exact tests. We did not correct for multiple comparisons across the five pilot tasks, but no difference was significant even without correction. The public benchmarks in the pilot may appear in the models' training data [30]. QuRE was published in 2025, which lowers this risk for the requirements results, but models trained later may have seen it.

Reliability. nimble gave identical results for one model version, but the version changed between our experiments. We never combined results from different versions, and we measured how much they differ.

7 Conclusion

A 9-billion-parameter decision model can judge weak words in requirements in context. It ranks defects well, misses almost none, and separates good from bad uses of the same word. Without any examples, it matches the best published result on QuRE. It runs in under a second per decision on local hardware. Its probabilities, however, are too high, so users need to choose a threshold on their own data. Its usefulness also depends strongly on the question: one rewording removed more than a quarter of its false alarms. Its apparent failure on two other datasets is explained by what their labels mean. In future work we will select thresholds on QuRE's development split, relabel a sample of ARTA following QuRE's guidelines, design a choice question for ambiguous pronouns, and evaluate more models and datasets with our tool.

Data Availability

Our evaluation tool, the exact questions with their hashes, the dataset converters and the logged decisions are available from the

author on request. QuRE and ReqEval are published under CC BY 4.0 and are available from their original sources. ARTA has no stated license, so we do not redistribute our converted version of it.

Acknowledgments

We used generative AI tools to implement parts of the evaluation tool and to draft and edit this manuscript. The author checked all analyses and claims and takes full responsibility for the content.

References

[1] Sallam Abualhaija. 2021. frieden84/nlp4re-reqeval: Second Release. Zenodo dataset (ReqEval shared task, 3rd NLP4RE Workshop at REFSQ 2020). Annotated by S. Abualhaija, F. Dalpiaz, A. Ferrari, X. Franch, and D. Fucci; see https://github.com/frieden84/nlp4re-reqeval. Accessed 2026-10-20. doi:10.5281/zenodo.4471411

[2] Sallam Abualhaija, F. Basak Aydemir, Fabiano Dalpiaz, Davide Dell'Anna, Alessio Ferrari, Xavier Franch, and Davide Fucci. 2024. Replication in Requirements Engineering: The NLP for RE Case. *ACM Transactions on Software Engineering and Methodology* 33, 6 (2024), 1–33. doi:10.1145/3658669

[3] Chetan Arora, John Grundy, and Mohamed Abdelrazek. 2024. Advancing Requirements Engineering Through Generative AI: Assessing the Role of LLMs. In *Generative AI for Effective Software Development*. Springer Nature Switzerland, 129–148. doi:10.1007/978-3-031-55642-5_6

[4] Daniel M. Berry, Erik Kamsties, and Michael M. Krieger. 2003. From Contract Drafting to Software Specification: Linguistic Sources of Ambiguity – A Handbook. University of Waterloo. Accessed 2026-10-20. https://cs.uwaterloo.ca/~dberry/handbook/ambiguityHandbook.pdf

[5] Bespoke Labs. 2026. Nimble: Public Benchmarks. https://github.com/bespokelabsai/nimble/blob/main/docs/PUBLIC_BENCHMARKS.md. Accessed 2026-10-20.

[6] Bespoke Labs, Shreyas Pimpalgaonkar, Pratik Dalal, and Maheswaran Sathiamoorthy. 2026. Nimble. https://github.com/bespokelabsai/nimble. Model: https://huggingface.co/bespokelabs/Bespoke-Nimble-9B. Accessed 2026-10-20. https://github.com/bespokelabsai/nimble

[7] Glenn W. Brier. 1950. Verification of Forecasts Expressed in Terms of Probability. *Monthly Weather Review* 78, 1 (1950), 1–3. doi:10.1175/1520-0493(1950)078<0001:VOFEIT>2.0.CO;2

[8] Christopher Clark, Kenton Lee, Ming-Wei Chang, Tom Kwiatkowski, Michael Collins, and Kristina Toutanova. 2019. BoolQ: Exploring the Surprising Difficulty of Natural Yes/No Questions. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Association for Computational Linguistics, 2924–2936. doi:10.18653/v1/N19-1300

[9] Bradley Efron. 1979. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics* 7, 1 (1979), 1–26. doi:10.1214/aos/1176344552

[10] Saad Ezzini, Sallam Abualhaija, Chetan Arora, and Mehrdad Sabetzadeh. 2022. Automated Handling of Anaphoric Ambiguity in Requirements: A Multi-Solution Study. In *Proceedings of the 44th International Conference on Software Engineering (ICSE)*. ACM, 187–199. doi:10.1145/3510003.3510157

[11] Henning Femmer, Frank Houdek, Max Unterbusch, and Andreas Vogelsang. 2025. Description and Comparative Analysis of QuRE: A New Industrial Requirements Quality Dataset. arXiv:2508.08868 doi:10.48550/arXiv.2508.08868

[12] Henning Femmer, Frank Houdek, Max Unterbusch, and Andreas Vogelsang. 2025. QuRE Dataset. Zenodo dataset. Accessed 2026-10-20. doi:10.5281/zenodo.16049822

[13] Henning Femmer, Daniel Méndez Fernández, Stefan Wagner, and Sebastian Eder. 2017. Rapid Quality Assurance with Requirements Smells. *Journal of Systems and Software* 123 (2017), 190–213. doi:10.1016/j.jss.2016.02.047

[14] Julian Frattini, Lloyd Montgomery, Davide Fucci, Michael Unterkalmsteiner, Daniel Mendez, and Jannik Fischbach. 2024. Requirements Quality Research Artifacts: Recovery, Analysis, and Management Guideline. *Journal of Systems and Software* 216 (2024), 112120. doi:10.1016/j.jss.2024.112120

[15] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. 2017. On Calibration of Modern Neural Networks. In *Proceedings of the 34th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 70)*, Doina Precup and Yee Whye Teh (Eds.). PMLR, 1321–1330. https://proceedings.mlr.press/v70/guo17a.html

[16] Arshia Hemmat, Mohammadreza Sharbaf, Shekoufeh Kolahdouz-Rahimi, Kevin Lano, and Sobhan Y. Tehrani. 2025. Research Directions for Using LLM in Software Requirement Engineering: A Systematic Review. *Frontiers in Computer Science* 7 (2025), 1519437. doi:10.3389/fcomp.2025.1519437

[17] ISO/IEC/IEEE. 2018. ISO/IEC/IEEE 29148:2018 Systems and Software Engineering – Life Cycle Processes – Requirements Engineering. International Standard. doi:10.1109/IEEESTD.2018.8559686

[18] Jennifer Krisch, Melanie Dick, Ronny Jauch, and Ulrich Heid. 2016. A Lexical Resource for the Identification of "Weak Words" in German Specification Documents. In *Proceedings of the Tenth International Conference on Language Resources and Evaluation (LREC 2016)*. European Language Resources Association (ELRA), Portorož, Slovenia, 2846–2850. https://aclanthology.org/L16-1454/

[19] Logan Lebanoff and Fei Liu. 2018. Automatic Detection of Vague Words and Sentences in Privacy Policies. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 3508–3517. doi:10.18653/v1/D18-1387

[20] Quinn McNemar. 1947. Note on the Sampling Error of the Difference Between Correlated Proportions or Percentages. *Psychometrika* 12, 2 (1947), 153–157. doi:10.1007/BF02295996

[21] Ollama. 2026. Decision. Ollama documentation. Accessed 2026-10-20. https://docs.ollama.com/capabilities/decision

[22] Ollama. 2026. Ollama now supports Jev-style decision models. Ollama blog, 29 September 2026. Accessed 2026-10-20. https://ollama.com/blog/ollama-now-supports-jev-style-decision-models

[23] Ollama. 2026. System One (API reference). Ollama documentation. Accessed 2026-10-20. https://docs.ollama.com/api/systemone

[24] Mahdi Pakdaman Naeini, Gregory Cooper, and Milos Hauskrecht. 2015. Obtaining Well Calibrated Probabilities Using Bayesian Binning. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 29. AAAI Press. doi:10.1609/aaai.v29i1.9602

[25] Qwen, An Yang, Baosong Yang, Beichen Zhang, Binyuan Hui, et al. 2024. Qwen2.5 Technical Report. arXiv:2412.15115 doi:10.48550/arXiv.2412.15115

[26] Qwen Team. 2026. Qwen3.5-9B. Hugging Face model card. Accessed 2026-10-20. https://huggingface.co/Qwen/Qwen3.5-9B

[27] Inioluwa Deborah Raji, Emily Denton, Emily M. Bender, Alex Hanna, and Amandalynne Paullada. 2021. AI and the Everything in the Whole Wide World Benchmark. In *Proceedings of the Neural Information Processing Systems Track on Datasets and Benchmarks*, Vol. 1. https://datasets-benchmarks-proceedings.neurips.cc/paper/2021/hash/084b6fbb10729ed4da8c3d3f5a3ae7c9-Abstract-round2.html

[28] Paul Ralph et al. 2020. Empirical Standards for Software Engineering Research. arXiv:2010.03525 doi:10.48550/arXiv.2010.03525

[29] Krishna Ronanki, Beatriz Cabrero-Daniel, and Christian Berger. 2023. ChatGPT as a Tool for User Story Quality Evaluation: Trustworthy Out of the Box?. In *Agile Processes in Software Engineering and Extreme Programming – Workshops (Lecture Notes in Business Information Processing)*. Springer Nature Switzerland, 173–181. doi:10.1007/978-3-031-48550-3_17

[30] Oscar Sainz, Jon Campos, Iker García-Ferrero, Julen Etxaniz, Oier Lopez de Lacalle, and Eneko Agirre. 2023. NLP Evaluation in trouble: On the Need to Measure LLM Data Contamination for each Benchmark. In *Findings of the Association for Computational Linguistics: EMNLP 2023*. Association for Computational Linguistics, 10776–10787. doi:10.18653/v1/2023.findings-emnlp.722

[31] June Sallou, Thomas Durieux, and Annibale Panichella. 2024. Breaking the Silence: the Threats of Using LLMs in Software Engineering. In *Proceedings of the 2024 ACM/IEEE 44th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. ACM, 102–106. doi:10.1145/3639476.3639764

[32] Richard Socher, Alex Perelygin, Jean Wu, Jason Chuang, Christopher D. Manning, Andrew Ng, and Christopher Potts. 2013. Recursive Deep Models for Semantic Compositionality Over a Sentiment Treebank. In *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 1631–1642. doi:10.18653/v1/D13-1170

[33] Abdullah Tahir, Nafiseh Kahani, Mohamed Nasr, Masoud Barati, and Paniz Oghabi. 2026. Software Requirements Ambiguity Resolution Using Large Language Models. In *2026 IEEE 34th International Requirements Engineering Conference Workshops (REW)*. IEEE, 84–93. doi:10.1109/REW72749.2026.00022

[34] Katherine Tian, Eric Mitchell, Allan Zhou, Archit Sharma, Rafael Rafailov, Huaxiu Yao, Chelsea Finn, and Christopher Manning. 2023. Just Ask for Calibration: Strategies for Eliciting Calibrated Confidence Scores from Language Models Fine-Tuned with Human Feedback. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, 5433–5442. doi:10.18653/v1/2023.emnlp-main.330

[35] Max Unterbusch and Andreas Vogelsang. 2026. Context-Adaptive Requirements Defect Prediction through Human-LLM Collaboration. In *Proceedings of the IEEE/ACM 48th International Conference on Software Engineering: New Ideas and Emerging Results (ICSE-NIER)*. ACM, 221–225. doi:10.1145/3786582.3786843

[36] Andreas Vogelsang and Jannik Fischbach. 2025. Using Large Language Models for Natural Language Processing Tasks in Requirements Engineering: A Systematic Guideline. In *Handbook on Natural Language Processing for Requirements Engineering*. Springer Nature Switzerland, 435–456. doi:10.1007/978-3-031-73143-3_16

[37] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (1927), 209–212. doi:10.1080/01621459.1927.10502953

[38] Claes Wohlin, Per Runeson, Martin Höst, Magnus C. Ohlsson, Björn Regnell, and Anders Wesslén. 2024. *Experimentation in Software Engineering*. Springer, Berlin, Heidelberg. doi:10.1007/978-3-662-69306-3

[39] Miao Xiong, Zhiyuan Hu, Xinyang Lu, Yifei Li, Jie Fu, Junxian He, and Bryan Hooi. 2024. Can LLMs Express Their Uncertainty? An Empirical Evaluation of Confidence Elicitation in LLMs. In *The Twelfth International Conference on Learning Representations (ICLR)*. https://openreview.net/forum?id=gjeQKFxFpZ

[40] Mohammad Amin Zadenoori, Jacek Dąbrowski, Waad Alhoshan, Liping Zhao, and Alessio Ferrari. 2025. Large Language Models (LLMs) for Requirements Engineering (RE): A Systematic Literature Review. arXiv:2509.11446 doi:10.48550/arXiv.2509.11446

[41] Morteza Zakeri-Nasrabadi. 2020. Automatic Requirements Testability Analysis Dataset. Zenodo dataset (ARTA). Accessed 2026-10-20. doi:10.5281/zenodo.4266727

[42] Morteza Zakeri-Nasrabadi and Saeed Parsa. 2024. Natural Language Requirements Testability Measurement Based on Requirement Smells. *Neural Computing and Applications* 36, 21 (2024), 13051–13085. doi:10.1007/s00521-024-09730-x

[43] Xiang Zhang, Junbo Zhao, and Yann LeCun. 2015. Character-level Convolutional Networks for Text Classification. In *Advances in Neural Information Processing Systems*, Vol. 28. https://papers.nips.cc/paper_files/paper/2015/hash/250cf8b51c773f3f8dc8b4be867a9a02-Abstract.html

[44] Liping Zhao, Waad Alhoshan, Alessio Ferrari, Keletso J. Letsholo, Muideen A. Ajagbe, Erol-Valeriu Chioasca, and Riza T. Batista-Navarro. 2021. Natural Language Processing for Requirements Engineering: A Systematic Mapping Study. *Comput. Surveys* 54, 3 (2021), 1–41. doi:10.1145/3444689